

UNIVERSITÉ SORBONNE PARIS NORD

IUT DE VILLETANEUSE

Département Informatique

SAÉ S2.01 – Base de la programmation objet

RAPPORT DE SYNTHÈSE :

Groupe MARS

Anass FATHI

Anis ROCHER

Destinataire : M. Lemaire

Sous la direction de : B. Lemaire & H. AZZAG

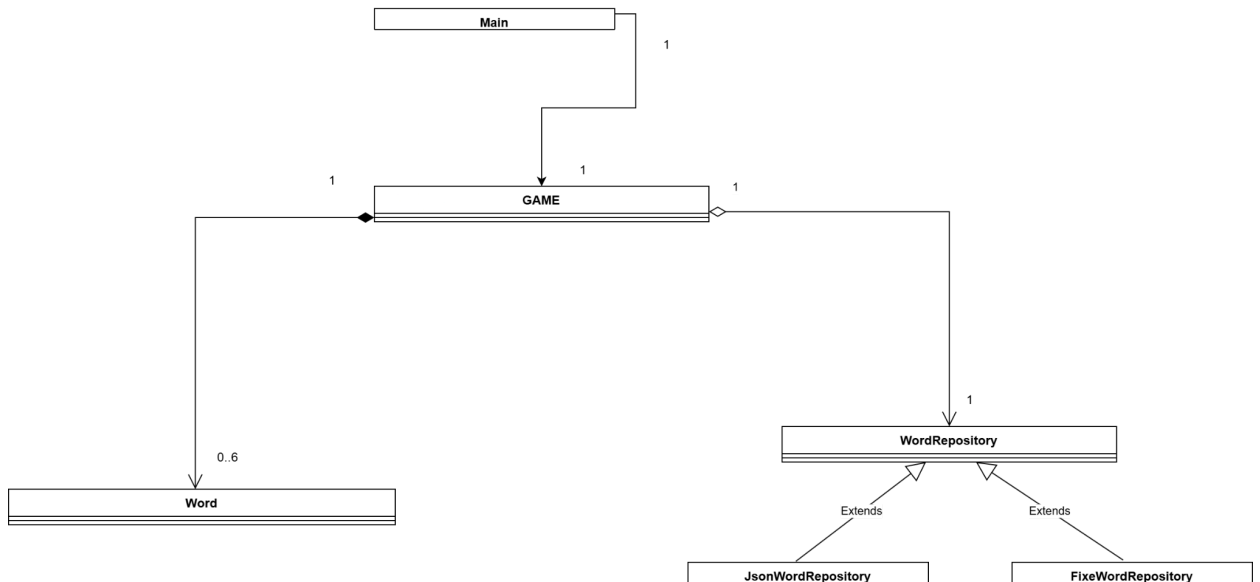
Date de rendu : 14/06/2026

Table des Matières

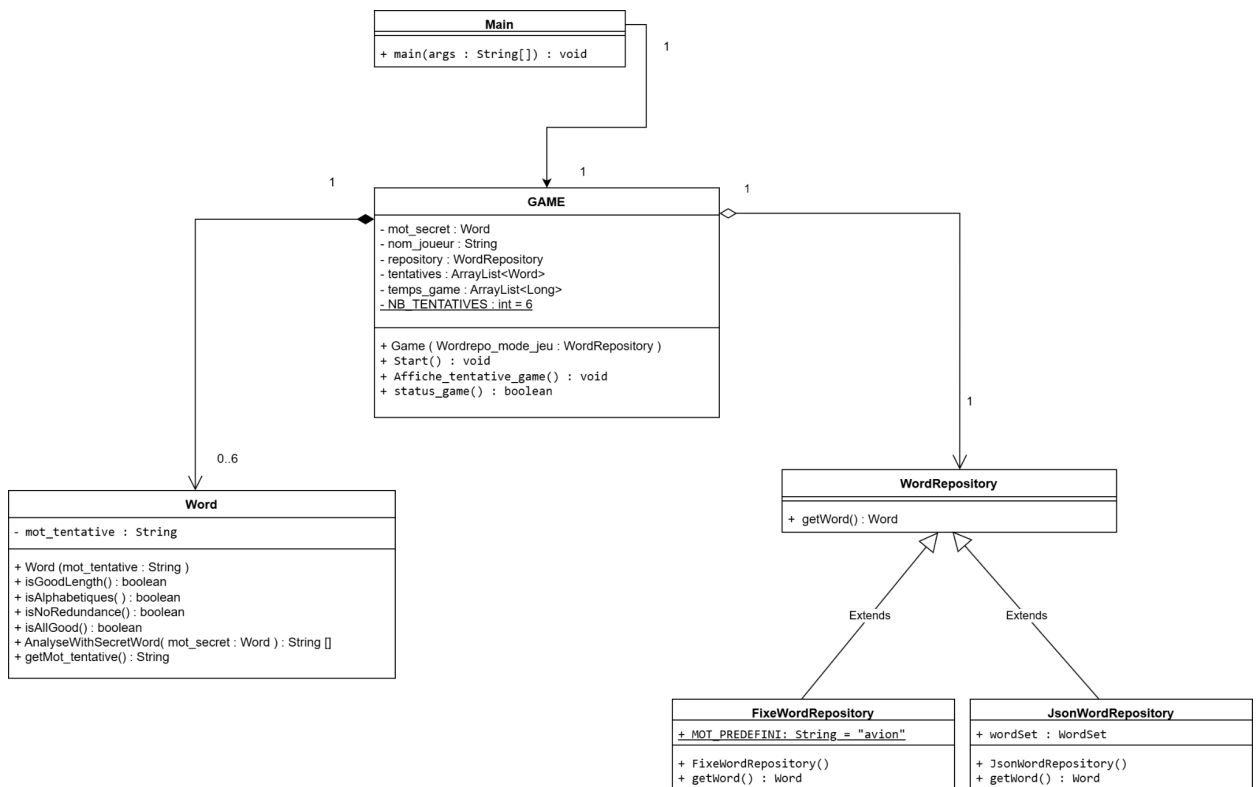
I. Modélisation Logicielle (UML)	3 page
1.1. Diagramme de classes UML simplifié	3 page
1.2. Diagramme de classes UML détaillé finale	3 page
1.3. Diagramme de classes UML détaillé première	4 page
1.4. Justification des choix liés au diagramme	4 page
II. Choix algorithmique et fonctionnalité optionnelle	5 page
2.1. Algorithme d'Analyse du Mot Secret	5 page
2.2. Fonctionnalités optionnelles	6 page
III. Cas de test officiels (Annexe B)	6 page
3.1. Procédure de lancement du programme	7 page
3.2. Validation des contraintes de saisie	8 page
3.3. Validation des scénarios de jeu	8 page
IV. Organisation du Travail et Bilan de la SAÉ	9 page
4.1. Organisation du travail en binôme	9 page
4.2. Difficultés rencontrées	9 page
V - Conclusion	10 page

I. Modélisation Logicielle (UML)

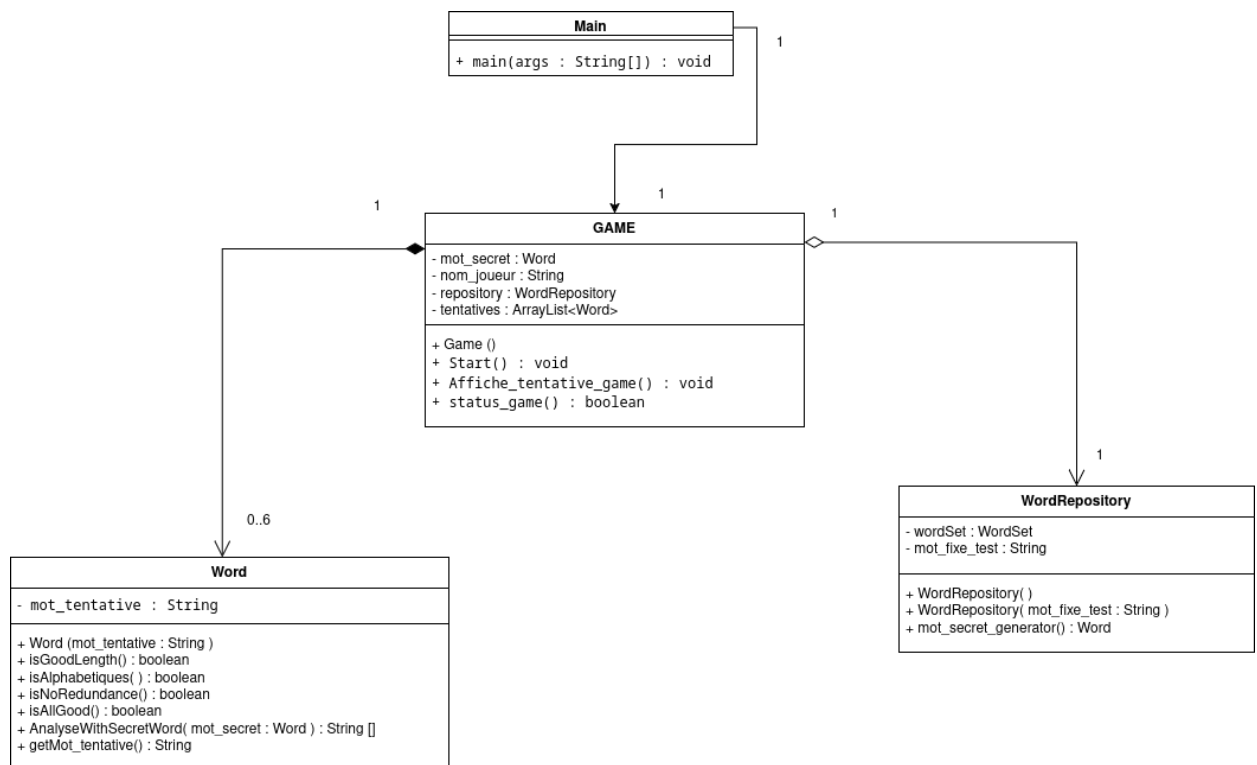
1.1. Diagramme de classes UML simplifié



1.2. Diagramme de classes UML détaillé final



1.3. Diagramme de classes UML détaillé premier



1.4. Justification de nos choix liées au diagramme

Notre architecture UML finale est représentée dans la section 1.2 : « Diagramme de classes UML détaillé final ». Cependant, grâce à un gain de temps dû à une bonne organisation, nous avons pu implémenter plusieurs fonctionnalités optionnelles. C'est pourquoi notre diagramme UML a dû être mis à jour en conséquence. Cette architecture se base sur la construction de six grandes classes nommées **Word**, **WordRepository**, **Game**, **Main**, **FixeWordRepository** et **JsonWordRepository**.

Débutons l'analyse de nos choix en commençant par la classe **WordRepository** et ses héritiers. Cette classe est directement reliée à la classe **Game** par une agrégation, en effet, tout objet **Game** contient un **WordRepository**. De plus, cette classe prend en attribut un **WordSet** venant de l'implémentation de votre bibliothèque avec **JsonWordSet**. Cette classe a été construite selon des directives abstraites et possède deux classes héritières.

JsonWordRepository, la première héritière, se charge du mode de jeu où l'on doit choisir un mot aléatoire, notamment en utilisant l'attribut **WordSet** et la bibliothèque prévue à cet effet. La seconde classe héritière, nommée **FixeWordRepository**, se charge, elle, du second mode de

jeu servant au test, avec pour mot secret « avion ». Enfin, toutes deux sont caractérisées par une méthode nommée `getWord`, nous constatons ici du polymorphisme.

Passons à la classe `Word`, qui représente l'une des classes mères de notre projet. En effet, celle-ci est reliée par composition à la classe `Game` et va servir à contrôler la bonne mise en œuvre des différentes règles métier, telles que la bonne taille du mot ou encore la non-redondance d'une lettre dans ce mot. Cette classe prend en attribut une chaîne de caractères qui sera transformée en objet `Word` par ces différentes règles métier. Nous pouvons porter notre attention sur l'une des méthodes de `Word` nommée `AnalyseWithSecretWord`, qui est une méthode primordiale au bon fonctionnement de notre jeu, puisqu'elle permet de comparer notre tentative à l'instant T à notre mot secret (qu'il soit choisi aléatoirement ou prédéfini pour nos tests). Nous nous attarderons plus en détail sur celle-ci.

Troisièmement la classe qui relie tous les Classe comme vu ci-dessus dans notre diagramme UML la classe `Game`. Cette classe est le cœur de notre jeu puisqu'elle permet de mettre en harmonie toutes les méthodes des autres classes ainsi que la mise en page si je puis dire du jeu, le côté graphique.

Venons à notre dernière Classe, celle du nom de `Main`. Comme son nom l'indique, celle-ci représente la classe qui va effectuer le lancement de notre partie. Elle assure le démarrage de l'application et l'initialisation du jeu en prenant soin de respecter les directives du joueur.

Enfin pour des questions de respect du sujet et de par les conseils de M.Lemaire, nous avons changé en conséquence du modelage du sujet SAE, l'implémentation des classe `FixeWordRepository` et `JsonWordRepository`. Par cet effet, notre diagramme a dû être réhabilité en conséquence, cependant pour des questions de possible suspicion de rétro-analyse nous avons décidé de mettre notre diagramme UML avant l'implémentation de ces classes et des fonctionnalités optionnelles que nous détaillerons plus bas. Celle-ci est représentée dans la partie 1.3. "Diagramme de classes UML détaillé premier".

II. Choix algorithmique et fonctionnalité optionnelle

2.1. Algorithme d'Analyse du Mot Secret

L'analyse entre la tentative du joueur à celle stocker en tant que mot secret représente l'une des méthodes primordiales à la réalisation de notre projet de jeu `Find My Word`. Cette méthode a pour sortie un tableau de chaîne de caractère. Cette méthode se caractérisait par un algorithme que l'on pourrait séparer sous 3 différentes parties.

Notre premier cas, la lettre de la tentative de notre joueur concorde avec celle du mot secret. Dans ce cas, la nôtre méthode stocke la réponse "OK" à la énième position à laquelle celle-ci est validée.

Notre second cas, lui, consiste à trouver une égalité de caractère entre le caractère i du joueur et le mot secret pour pouvoir attester de la validité d'un caractère dans la chaîne de caractère du mot secret. Celui-ci stocke sous la valeur "PRESENT" .

Enfin, notre troisième cas, découlant du second, est le cas où nous ne trouvons aucune similarité entre le caractère de la tentative du joueur et celui du mot secret. Ce cas se caractérise par le stockage de la réponse "ABSENT".

Cet algorithme nous permet d'analyser la tentative du joueur à celle du mot secret tout en respectant les contraintes métiers appliquées par le sujet à savoir que le mot contient exactement 5 lettres, qu'il ne contient que des caractères alphabétiques et qu'il ne contient aucune lettre répétée.

2.2. Fonctionnalités Optionnelles

Notre travail en équipe nous a permis de pouvoir nous répartir les tâches de façon claire et efficace. Cette organisation nous a permis de pouvoir développer quelques fonctionnalités optionnelles qui sera, nous l'espérons, appréciée du jury.

Nos fonctionnalités optionnelles se concrétisent tout d'abord par l'implémentation d'une boucle de jeu à savoir le fait de pouvoir à la fin de chaque partie avoir le droit de relancer une nouvelle partie avec choix du mode inclus. Ces fonctionnalités ne s'arrêtent pas seulement à cela, mais également par l'implémentation d'un timer qui chronomètre chacune de vos tentatives et les temps global de la partie, une fonctionnalité qui nous a été tenue à cœur de mettre pour ajouter un défi en plus à ce jeu.

Enfin, notre dernière fonctionnalité se présente par le choix au lancement du jeu ou lorsqu'une partie se termine de pouvoir choisir le mode de jeu pour relancer la partie. Les différents modes de jeu sont le mode avec un mot prédéfini et le second le mode avec un mot aléatoire en tant que mot secret.

III. Cas de test officiels (Annexe B)

3.1. Procédure de lancement du programme

Pour lancer notre jeu dans les meilleures conditions, je vais détailler dans cette partie les procédures de lancement et les précommandation avant lancement .

Tout d'abord notre jeu utilise une bibliothèque de fabrication M.Lemaire de ce fait vous devez être requis de JAVA SE 11 ou d'une version plus moderne ,de plus le jar compris dans le zip est compilé pour JAVA 11 (bytecode version 55) .

Une fois le dossier décompressé, rendez vous dans le répertoire principale de celui-ci, si cette étape a été correctement effectuée vous devriez être en présence de 3 dossiers (data , src , jar) pour voir cela sous environnement linux tapez la commande ls et sous environnement windows dir . Nous allons maintenant passer à une étape clé pour le bon lancement de notre jeu nommée la compilation . Pour cela, veuillez taper la commande .

Distribution Linux : `javac -cp ".:jar/wordset.jar" -d . src/*.java`

Windows : `javac -cp ".;jar/wordset.jar" -d . src/*.java`

Vous êtes arrivé à l'étape final pour pouvoir lancer le jeu .Il ne vous manque plus qu'à lancer le jeu avec :

distribution Linux : `java -cp ".:jar/wordset.jar" words.Main`

Windows : `java -cp ".;jar/wordset.jar" words.Main`

Si vous ne voulez plus jouer a notre très bon jeu et le désinstaller suivez cette procédure toujours dans le même répertoire que précédemment :

distribution Linux :

```
cd ..  
rm -R Anass_Anis_FindMyWord  
rm -R Anass_Anis_FindMyWord.zip
```

Windows :

```
cd ..  
rmdir /s /q Anass_Anis_FindMyWord  
del /f /q Anass_Anis_FindMyWord.zip
```

3.2. Validation des contraintes de saisie

Pour garantir le respect des contraintes du jeu, l'application vérifie et valide chaque proposition du joueur avant de l'exécuter. Ces vérifications sont faites grâce à la méthode `isAllGood()` de la class `Word` qui vérifie chaque cas possible en appelant différentes méthodes interne:

- `isGoodLength()`: Elle vérifie que le mot fait bien 5 lettres. Si le joueur saisit un mot trop court ou trop long, alors la saisie est rejetée.
- `isAlphabetiques()`: Elle vérifie à l'aide d'une expression régulière (Regex) que le mot contient uniquement des lettres et rien d'autre.
- `isNoRedundance()`: Elle vérifie qu'aucune lettre ne se répète dans le mot (comme dans "belle" par exemple).

Si lors de l'appel de la méthode `isAllGood()` l'une de ces validations échoue, alors le programme affiche un message d'erreur qui l'invite à réessayer sans compter la tentative comme un essai manqué.

3.3. Validation des scénarios de jeu

Une fois la saisie du joueur validée par la classe `Word`, la classe `Game` prend le relais pour appliquer toutes les règles qui concernent les tours de jeu ainsi que la détection des scénarios de fin de partie. Nous avons testé et validé l'ensemble des cas écrit dans l'annexe B du fichier `SAE_FindMyWord_v_1.2.md` :

- **Analyse de la position des lettres (Cas 5 à 8):** La méthode `AnalyseWithSecretWord()` de la classe `Word` compare la proposition du joueur au mot secret et retourne un tableau de chaînes de caractère contenant soit "OK" si la lettre est correct est à la bonne place, "PRESENT" si la lettre est présente dans le mot secret mais mal placée ou "ABSENT" si la lettre n'est pas présente dans le mot secret.
- **Affichage de la grille (Cas 9) :** A chaque tour validé, la méthode `Affiche_tentative_game()` met à jour la console. Elle affiche les lettres saisies en majuscules entouré de crochets ainsi que le résultat de l'analyse, ce qui permet d'avoir un historique visuel des mots saisis et de leurs analyses.
- **Scénario de Victoire (Cas 11):** La méthode `status_game()` vérifie après chaque saisie si le dernier mot proposé est identique au mot secret. Si c'est le cas, la boucle s'arrête, le message "Vous avez GAGNE !!!!" est affiché dans la console ainsi que le temps de la partie calculé en secondes.
- **Scénario de Défaite (Cas 12):** Si la boucle `for` atteint la limite de 6 tentatives sans que le mot soit découvert, le jeu affiche le message "PERDU : Vous avez utilisé tous vos essais", et il affiche le mot secret ainsi que le temps de la partie calculé en secondes.
- **Gestion de fin de partie :** Lors de la fin d'une partie (victoire ou défaite), la classe `Main` propose de relancer une partie de façon interactive avec un menu [o / n], permettant au joueur de directement relancer une partie sans fermer la console.

IV. Organisation du Travail et Bilan de la SAÉ

4.1. Organisation du travail en binôme

Pour cette SAE notre objectif a été de trouver un équilibre pour que chacun fasse autant sur l'ensemble du projet que ça soit pour le développement du code ou pour la rédaction du fichier pdf ainsi que pour le diagramme UML.

Tout d'abord, nous avons travaillé ensemble sur la conception du premier diagramme de classe UML. Cela nous a permis de bien comprendre l'architecture de l'application et de définir la répartition des tâches. Grâce à cette analyse de la représentation UML, nous avons séparé le code en deux parties pour pouvoir travailler en même temps.

Anis a pris en charge la classe Word.java, il a coder toutes les fonctions de validation (taille, alphabet et non redondance)

Anass a travaillé en parallèle sur la récupération de mots. En suivant l'architecture demandée, il a écrit notre propre code dans la classe abstraite WordRepository et dans les classes JsonWordRepository et FixeWordRepository. Il a coder la logique interne pour aller lire le fichier mots.json.

Pour assembler tout ça, on s'est mis ensemble pour coder les classes Game.java et Main.java. Pour s'échanger nos fichiers on a utilisé GitHub. On a créé un dépôt et on faisait des "commits" et des "pushs" réguliers de nos avancées.

Enfin, comme le contrôle de SAE se fait individuellement, nous avons chacun pris le temps à la fin du projet pour relire ensemble tout le code.

4.2. Difficultés rencontrées

Au cours de cette SAE, notre binôme a dû faire face à deux grosses difficultés:

- **La logique de la méthode AnalyseWithSecretWord() dans la classe Word.java** : Cette méthode a été compliquée à penser à cause de toutes les conditions à prendre en compte en même temps. Le plus dur a été de gérer les boucles, par exemple pour que dans la première boucle on vérifie si la lettre est à la bonne position et si cette condition n'est pas validé alors on lance une nouvelle boucle dans la boucle pour vérifier si la lettre est présente dans le mot. Cela n'a pas été facile à penser au début et il nous a fallu du temps pour réussir à coder cette méthode.

- **L'imbrication des méthodes dans Start() dans la classe Game.java** : Dans la méthode Start(), il a fallu imbriquer plusieurs méthodes venant de classes différentes. De plus, il fallait gérer l'affichage graphique dans la console, tout en appelant au bon moment les méthodes de validation de la classe Word et en calculant le temps avec le chronomètre. Réussir à lier tout cela proprement dans une seule fonction a été une vraie difficulté pour nous.

V - Conclusion

Pour conclure ce projet , nous avons apprécié faire ce projet d'une part car elle nous a permis de mettre en action nos compétences et connaissances acquises en cours tels que le polymorphisme ou encore l'utilisation des classes abstraites . D'autre part, le travail en équipe nous a aussi permis d'acquérir une réelle rigueur concernant notre organisation .